Task 1 — Design Your Own Data Model and Blueprint Class

Description:

Students will explore the concept of **classes**, **constructors**, and **object instances** in Kotlin. They'll design and document a data model (blueprint) for an app feature — such as an e-commerce product, contact card, task tracker item, or recipe. The focus is on **defining meaningful properties, encapsulation, and methods**, then planning how this class could be integrated with a Compose UI.

Practical Action Items:

1. **Concept Review & Notes:**
   - Define in your own words:
     - What is a *class* in Kotlin?
     - Difference between a *class* and *object*?
     - What are *primary* and *secondary* constructors?
     - Purpose of *init blocks* and *member functions*.
   - Give one short Kotlin example of a simple data class.
2. **Design Your Model:**
   - Pick one domain (e.g., **Fitness App Workout**, **Recipe Manager**, **Student Tracker**, **Budget App**).
   - Create a class design with:
     - At least **4 properties** (String, Int, Boolean, List, etc.)
     - At least **2 functions/methods** (e.g., calculate total calories, format address, apply discount).
     - One **init block** printing or validating something."

- Example outline:

```kotlin
class Recipe(
    val name: String,
    val ingredients: List<String>,
    val cookingTime: Int,
    var isFavorite: Boolean = false
) {
    init {
        println("Recipe created: $name")
    }

    fun estimatedEffort(): String =
        if (cookingTime < 20) "Easy" else "Medium/Hard"

    fun toggleFavorite() { isFavorite = !isFavorite }
}
```

3. Diagram and Object Instances:

- Draw a simple UML-style box diagram showing properties & methods.
- Create **2 object instances** of your class with example data.
- Demonstrate a small Kotlin snippet showing how you might call the functions on those objects.

4. Compose Integration Plan:

- Write 3–4 lines on how you could display these object instances in a Compose UI (for example, a list of recipes with favorite buttons).

5. Reflection:

- 4–5 lines on what you learned designing a class.
- Why modeling data early helps you write cleaner UI and logic code.

Submission Options:

- PDF/Word file named YourName_Task1_ClassDesign.pdf containing:
  - Notes, class design, object examples, diagram or screenshot, integration plan, reflection.
- Optional: Link to a code repo with your Kotlin file.

Outcome:

Students will understand how to model real-world entities in code using classes, create reusable blueprints, and plan for their use in a Compose UI.

Task 2 — Functional Programming with Lambdas and Higher-Order Functions

Description:

Students will explore **function types**, **lambdas**, and **higher-order functions** — powerful tools for cleaner Kotlin code. They'll plan a real-world example of reusing logic through higher-order functions and document how Compose uses lambdas (for example, onClick = { }).

Practical Action Items:

1. **Concept Notes:**
   - Define the following:
     - *Higher-order function*
     - *Lambda expression*
     - *Function type* (e.g., (Int, Int) -> Int)
     - *Inline functions* (briefly)
   - Write one example each:
     - A lambda that filters a list of integers.
     - A higher-order function that accepts a lambda as a parameter.
2. **Design a Mini Use Case:**
   - Choose a scenario such as **Expense filtering**, **Student grade evaluation**, or **Task sorting**.
   - Example idea: *Given a list of tasks, create a higher-order function that takes a lambda to determine which tasks are urgent.*
   - Define the data model (simple class or data class).
   - Write pseudo-code showing how you'd apply your lambda to a list.

3. **Compose Lambda Usage Research:**

   - Identify 3 places in Compose where lambdas are common (e.g., Button(onClick = { }), Modifier.clickable { } …).
   - For one of them, describe what parameters or states are being passed.
   - Write your own example Compose snippet:

     Button(onClick = { total++ }) { Text("Add") }z

4. **Function Type Exercise:**

   - Define a function type alias (e.g., typealias DiscountRule = (Double) -> Double) and show how you might pass different lambdas implementing that type to a calculation function.
   - Document what advantage this brings compared to hard-coding logic.

5. **Reflection:**

   - Discuss how using lambdas improves **code reusability** and **readability**.
   - How do lambdas simplify event handling in Compose?

Submission Options:

- PDF/Word file named YourName_Task2_Lambdas.pdf with notes, pseudo-code, examples, Compose snippet, reflection.
- Optional GitHub link to sample Kotlin file.

Outcome:

Students will understand functional programming principles in Kotlin, learn to apply higher-order functions and lambdas to simplify logic, and connect those concepts to Compose UI callbacks.

Task 3 — Mini Project: Compose App Feature Combining OOP and Lambdas

Description:

This final task integrates both **OOP concepts** and **functional programming**. Students will conceptualize a feature that combines a class model, list of objects, and lambda-based interactivity in Compose. It should be more complex than any previous session feature.

Practical Action Items:

1. **Choose Your App Theme:**
   - Ideas:
     - *Book Library* (list of Book objects with filter by genre lambda)
     - *Habit Tracker* (list of Habit objects with completion toggle lambda)
     - *E-Commerce Cart* (list of Product objects with discount function type)
   - Write a short description (3–4 lines) explaining your chosen theme.
2. **Design Class Structure and Data:**
   - Define one main class (e.g., Product, Book, Habit) with at least:
     - 4 properties,
     - 2 methods, and
     - a companion object or constant.
   - Create a list of 3–5 objects using this class.
   - Write pseudo-code or Kotlin snippet for a function that takes a lambda to filter or transform that list.

MindMatrix

3. **Compose UI Integration Plan:**
    - Sketch UI with LazyColumn or Row to display the object list.
    - Include interactive element (e.g., button using lambda to toggle favorite state).
    - Annotate how your UI responds to changes in object state.
4. **Flow Diagram and Architecture:**
    - Create a diagram showing:
        - Data class → Lambda filter → UI update.
    - Label which parts handle state and which perform business logic.
5. **Error & Edge Case Planning:**
    - Write at least 3 possible edge cases (e.g., empty list, null object, user input error).
    - Document how each is handled (safe call ?., default lambda, error message).
6. **Reflection & Improvement Plan:**
    - Reflect on how you combined OOP and lambdas.
    - What trade-offs did you find between object-oriented and functional styles?
    - What would you improve if turning this into a real app feature?

Submission Options:

- Word/PDF named YourName_Task3_IntegratedFeature.pdf including: class design, object list, lambda logic, Compose UI plan, diagram, edge cases, reflection.
- Optional GitHub link with working Compose feature.

Outcome:

Students will synthesize their understanding of object design and functional thinking to produce a more complete feature blueprint for a Compose app.
This mirrors real Android development where data models, state management, and UI interactions all work together.